Workgroup:	Network Working Group
Internet-Draft:	draft-narvaneni-agent-uri-05
Published:	20 October 2026
Intended Status:	Standards Track
Expires:	23 April 2027
Authors:	Y. Narvaneni *Independent Researcher*	S. P. Ravi *Independent Researcher*
---	---	---

The agent URI Scheme and Verifiable Agent Identity

Abstract

Software agents increasingly act on the Internet on behalf of people and organizations: they call APIs, browse websites, and delegate work to other agents. Protocols such as A2A and MCP define how agents communicate, but there is no interoperable way to answer a prior question: *which agent is this, and how can its claims be verified?*

This document defines the `agent` URI scheme as a stable, human-readable identifier for an agent, a deterministic mapping from that identifier to an HTTPS-hosted Agent Identity Document, anchoring of the agent's keys to a web origin or a Decentralized Identifier, signed statements that bind the agent to its protocol metadata, and a profile of HTTP Message Signatures for automated traffic that lets an agent prove its identity on HTTP requests. It deliberately does not define how agents communicate, describe their capabilities, or are invoked.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 23 April 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (https://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction 4
   1.1. Design Principles 4
   1.2. Non-Goals 5
2. Conventions and Terminology 5
3. Overview 6
4. The agent URI Scheme 6
   4.1. Syntax 6
   4.2. Normalization and Comparison 7
   4.3. Dereferencing 8
   4.4. Fragments 8
5. Agent Identity Document 8
   5.1. Format 8
   5.2. Key Identifiers 9
   5.3. Key Purposes 9
   5.4. Links 10
   5.5. Example 10
   5.6. Retrieval and Validation 11
6. Anchoring 12
   6.1. Origin-Anchored Agents 12
   6.2. DID-Anchored Agents 12
   6.3. Lifecycle, Rotation, and Revocation 13
7. Agent Statements 13
   7.1. Verification 13

8. Request Signing 14
8.1. Signing 14
8.2. Verification 15
9. Use with OAuth 15
10. Discovery 16
11. Security Considerations 17
11.1. Trust Model 17
11.2. Handle Reassignment 17
11.3. Operator Claims 17
11.4. Verifier-Initiated Fetches 17
11.5. Request Content and Replay 18
11.6. Key Compromise 18
11.7. Cross-Origin Metadata 18
11.8. Confusable Names 18
12. Privacy Considerations 18
13. IANA Considerations 19
13.1. URI Scheme Registration 19
13.2. Well-Known URI Registration 19
13.3. Media Types 20
13.3.1. application/agent-id+json 20
13.3.2. application/agent-statement+jwt 20
13.4. Signature-Agent Type 21
13.5. Link Relation 21
13.6. JSON Web Key Parameter 21
13.7. JSON Web Token Claim 22
14. References 22
14.1. Normative References 22
14.2. Informative References 24
Appendix A. Test Vectors: Agent URI Processing 25
A.1. Parsing and Canonicalization 25

A.2. Mapping to AID URLs 26
A.3. Reverse Mapping of Signature-Agent Values 27
A.4. AID Origin Comparison 27
Appendix B. Open Issues 28
Appendix C. Changes from draft-narvaneni-agent-uri-04 28
Acknowledgments 29
Authors' Addresses 29

1. Introduction

Agents are now first-class actors on the Internet. A single user task can cause an agent to call dozens of APIs and other agents, each of which needs to decide whether to serve the request. Today those services can see a TLS client, an IP address, perhaps an OAuth access token issued to *some* client, and a User-Agent string that anyone can forge. They cannot reliably tell which agent is calling, or whether the protocol metadata it advertises was really published by whoever controls it.

Communication protocols already exist. [A2A] defines agent-to-agent interaction and an Agent Card; [MCP] defines how agents reach tools and publishes server metadata; OpenAPI describes HTTP APIs. Each of these describes *how to talk to* an endpoint. None of them defines a portable answer to *who* an agent is that works across all of them.

This document defines that missing identity layer:

1. **A name.** The agent URI (e.g., `agent://example.com/planner`) is a stable, human-readable identifier for an agent (Section 4).
2. **A document.** Every agent URI maps to an HTTPS URL that serves an Agent Identity Document listing the agent's claimed operator, public keys, and links to its protocol metadata (Section 5).
3. **An anchor.** The agent's keys are trusted because of where the document is served, or because of a Decentralized Identifier bound to the agent URI (Section 6).
4. **Statements.** The agent signs statements binding itself to specific protocol metadata, so that metadata can be hosted anywhere (Section 7).
5. **Proof on requests.** The agent signs the HTTP requests it makes (Section 8).

1.1. Design Principles

- **Identify, do not invoke.** An agent URI names an agent. It is not a locator for an endpoint and carries no invocation parameters.
- **One way to do each thing.** There is one mapping from name to document, one key identifier, and one way for an agent to prove its identity on a request.
- **Reuse before invent.** Every mechanism is a profile of an existing or in-progress standard: HTTPS and well-known URIs [RFC8615], JWK [RFC7517], JWS [RFC7515], HTTP Message Signatures [RFC9421] as profiled by [WEB-BOT-AUTH], and DIDs [DID-CORE].

- **Anchor trust outside the document being verified.** A key is trusted because of the origin or DID it is published under, never because a document names it about itself.
- **Authenticate keys, not claims.** Verification establishes control of keys anchored to an agent URI. Everything else an agent says about itself, including who operates it, remains a claim.

1.2. Non-Goals

The following are out of scope:

- Communication semantics, message formats, task lifecycles, and invocation. These belong to A2A, MCP, OpenAPI-described APIs, and similar protocols.
- Capability, skill, or tool description. Protocol metadata documents (for example, an A2A Agent Card or an MCP server card) remain authoritative for these.
- Credentials for short-lived agent instances, and bindings of agent keys to OAuth client authentication. These are expected to be specified in companion documents.
- Verification of organizations. This document does not verify that the operator an agent claims is real or actually operates it.
- Search, ranking, registries, reputation, pricing, and payments.
- Authorization policy. Knowing who an agent is does not grant it anything.

2. Conventions and Terminology

The key words "**MUST**", "**MUST NOT**", "**REQUIRED**", "**SHALL**", "**SHALL NOT**", "**SHOULD**", "**SHOULD NOT**", "**RECOMMENDED**", "**NOT RECOMMENDED**", "**MAY**", and "**OPTIONAL**" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

This document uses the terms "JSON object", "member", and "string" from [RFC8259], and "Dictionary", "Item", and "String" from [RFC9651].

Agent: A software system that acts autonomously or semi-autonomously, typically on behalf of a person or organization.

Operator: The person or organization that an agent claims is accountable for it.

Agent URI: An agent URI as defined in Section 4. The human-readable name of an agent.

Agent Identity Document (AID): A JSON document, retrieved by dereferencing an agent URI, that lists the agent's claimed operator, keys, and links (Section 5).

AID URL: The HTTPS URL that an agent URI maps to (Section 4.3).

AID Origin: The tuple ("https", host, 443) for the host of the agent URI.

Agent Key: A public key anchored to an agent (Section 6).

Key Identifier: The JWK SHA-256 Thumbprint of an Agent Key (Section 5.2).

Verifier: Any party that checks an agent's identity: a website or API receiving requests, an authorization server, a registry, or another agent.

3. Overview

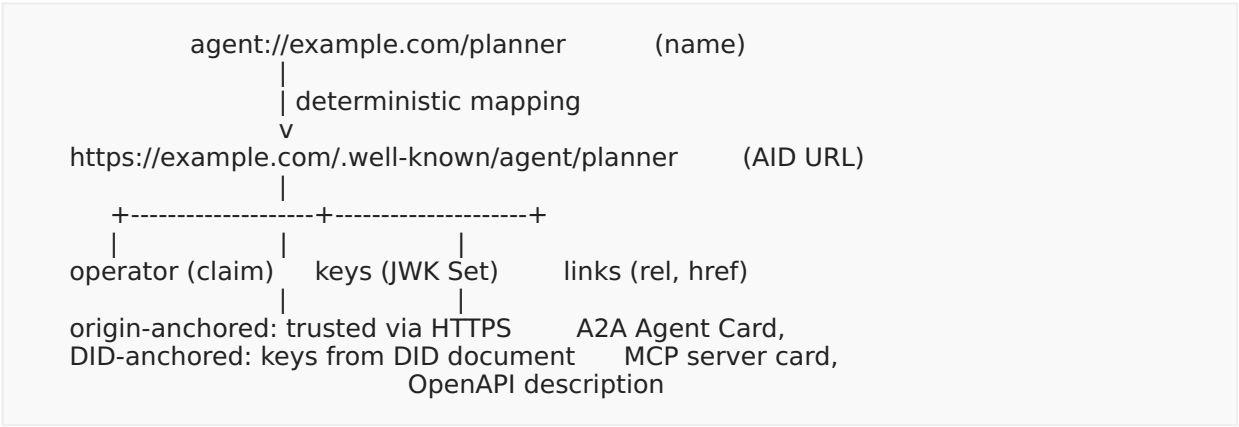


Figure 1: From agent URI to verifiable identity

A typical verification, for a request received by an API:

1. The request carries an HTTP message signature and a Signature-Agent member whose value is the AID URL of agent://example.com/planner.
2. The verifier derives the agent URI from the AID URL and retrieves the AID (or uses a cached copy).
3. The verifier finds the anchored key identified by the signature and verifies the signature and the request content.
4. The verifier now knows that the request was made by a holder of a key anchored to agent://example.com/planner. It also has the operator the agent *claims*, which it must not treat as verified (Section 11.3). Whether to serve the request remains a policy decision.

4. The agent URI Scheme

4.1. Syntax

The following ABNF [RFC5234] uses fragment from [RFC3986] and ALPHA and DIGIT from [RFC5234].

```
agent-uri    = "agent://" dns-name [ name-path ] [ "#" fragment ]

dns-name     = label 1*( "." label )
label        = let-dig [ *61( let-dig / "-" ) let-dig ]
let-dig      = ALPHA / DIGIT

name-path    = 1*8( "/" name-segment )
name-segment = lc-let-dig [ *61( lc-let-dig / "-" ) lc-let-dig ]
lc-let-dig   = %x61-7A / DIGIT        ; a-z, 0-9
```

Figure 2: ABNF for the agent URI

Additional constraints:

- dns-name **MUST NOT** exceed 253 octets, and its final label **MUST** contain at least one ALPHA character. Together with the ABNF, this excludes IPv4 addresses in dotted-decimal form and single-label names such as localhost.
- Internationalized domain names **MUST** be represented as A-labels [RFC5890] [RFC5891]. IP literals, userinfo, ports, and queries are not permitted.
- An agent URI with no name-path (e.g., agent://planner.example.com) identifies the domain's root agent.

Examples:

- agent://example.com/planner
- agent://planner.example.com
- agent://example.com/acme/billing-assistant
- agent://example.com/planner#kPrK_qmxVWaYVA9wwBF6Iuo3vVzz7TxHCTwXBygrS4k (a reference to one of the agent's keys; see Section 4.4)

4.2. Normalization and Comparison

The canonical form of an agent URI is produced by:

1. Lowercasing the scheme and dns-name.
2. Converting any U-labels to A-labels.
3. Removing a ":443" port, or a ":" with an empty port, after the host.
4. Removing a single trailing "/" if present.

Parsers **MAY** accept input that differs from the ABNF only by the deviations corrected above, and **MUST** normalize it. Any other deviation (another port, uppercase in name-path, percent-encoding, dot-segments, a query, a trailing "." on the host) **MUST** be rejected.

Two agent URIs identify the same agent if and only if their canonical forms, with any fragment removed, are identical octet strings. In particular, agent://example.com/planner and agent://planner.example.com are different agents, even if the same operator runs both.

Agent URIs in AIDs, statements, and tokens **MUST** be in canonical form and **MUST NOT** contain a fragment, unless a fragment is explicitly permitted.

4.3. Dereferencing

An agent URI is dereferenced by mapping it to its AID URL:

```
https://<dns-name>/.well-known/agent<name-path>      (named agent)
https://<dns-name>/.well-known/agent/_               (root agent)
```

For example:

Agent URI	AID URL
agent://example.com/planner	https://example.com/.well-known/agent/planner
agent://example.com/acme/billing-assistant	https://example.com/.well-known/agent/acme/billing-assistant
agent://planner.example.com	https://planner.example.com/.well-known/agent/_

Table 1: Agent URI to AID URL mapping

The root agent uses the reserved segment `_`, which cannot appear in a `name-segment`. This allows the root AID and named AIDs to coexist on static file hosting, so an operator needs nothing more than the ability to serve static files over HTTPS for the domain.

The mapping is purely syntactic and reversible: an HTTPS URL is an AID URL if and only if it has no port, query, or fragment and applying the reverse mapping yields an agent URI that is valid and canonical. No registry or index needs to be fetched to dereference an agent URI.

4.4. Fragments

A fragment in an agent URI identifies an Agent Key by its Key Identifier (Section 5.2). No other fragment semantics are defined.

5. Agent Identity Document

5.1. Format

An AID is a JSON object [RFC8259] served with media type `application/agent-id+json` (Section 13.3). It has the following members:

id: **REQUIRED**. String. The canonical agent URI that this document describes.

`operator`: **REQUIRED**. Object stating who the agent claims is accountable for it, with members `name` (**REQUIRED**, string) and `url` (**OPTIONAL**, string, HTTPS URL). These values are self-asserted (Section 11.3).

keys: **REQUIRED** for origin-anchored agents; **MUST** be absent for DID-anchored agents. A JWK Set [RFC7517] containing the agent's public keys (Section 6.1).

did: **OPTIONAL**. String. A DID [DID-CORE] whose DID document is the anchor for this agent's keys (Section 6.2).

name: **OPTIONAL**. String. A display name for people. It has no identity semantics.

status: **OPTIONAL**. String. One of `"active"` (default), `"suspended"`, or `"retired"` (Section 6.3).

links: **OPTIONAL**. Array of link objects (Section 5.4).

updated: **OPTIONAL**. String. The time the document was last changed, as an RFC 3339 [RFC3339] timestamp.

Members not defined here **MUST** be ignored by verifiers that do not understand them. Extension member names **SHOULD** be namespaced (e.g., `"acme_tier"`) to avoid collision with future versions of this document.

An AID **MUST NOT** exceed 64 KiB.

5.2. Key Identifiers

Every Agent Key is identified by its **Key Identifier**: the base64url-encoded JWK SHA-256 Thumbprint [RFC7638] of the key, computed as specified in [RFC7638] for EC keys and in Appendix A.3 of [RFC8037] for OKP keys. This is the identifier [WEB-BOT-AUTH] uses for `keyid`. It is used everywhere a key is named in this document: JWK `kid` values in an AID, agent URI fragments, the JWS `kid` header of statements, and the `keyid` parameter of request signatures.

Verifiers **MUST** compute the thumbprint of each candidate key themselves and select keys by comparing it with the presented Key Identifier. They **MUST NOT** rely on an operator-supplied `kid` or DID verification method `id` to select a key. An AID or DID document in which two keys have the same Key Identifier **MUST** be rejected.

For DID-anchored agents, each verification method is converted to a JWK before the thumbprint is computed:

- a `publicKeyJwk` value is used as-is (private members **MUST NOT** be present);
- a `publicKeyMultibase` value that is a Multikey encoding of an Ed25519 public key (base58btc with the `0xed01` header, as defined in [CID]) is converted to an OKP JWK with `"crv": "Ed25519"`; and
- verification methods in any other representation **MAY** be supported; those a verifier cannot convert are ignored.

5.3. Key Purposes

Agent Keys have two purposes: **assertion** (signing statements, Section 7) and **authentication** (signing HTTP requests, Section 8).

For DID-anchored agents, a key is usable for assertion only if the DID document's `assertionMethod` relationship references it, and for authentication only if `authentication` references it [DID-CORE].

For origin-anchored agents, a JWK **MAY** carry an `agent_use` member (Section 13.6): an array containing `"assertion"`, `"authentication"`, or both. A key without `agent_use` is usable for both purposes. A key whose `agent_use` is not an array of strings, or contains no recognized value, **MUST** be ignored.

Verifiers **MUST** enforce key purposes. Operators **SHOULD** give every key a single purpose, so that a key held by a running agent for signing requests cannot sign statements (Section 11.6).

5.4. Links

Each link object has the following members, with the semantics of Web Linking [RFC8288]:

`rel`: **REQUIRED**. String. A registered link relation type or a URI.

`href`: **REQUIRED**. String. An absolute HTTPS URL.

`type`: **OPTIONAL**. String. The media type of the target.

`statement`: **OPTIONAL**. String. An Agent Statement (Section 7) about the target.

Link relations expected to be common include:

Relation	Target
agent-card	The agent's [A2A] Agent Card
mcp-server-card	The agent's [MCP] server metadata
service-desc	An OpenAPI or other API description [RFC9727]

Table 2: Common link relations

For origin-anchored agents, a link whose `href` has the same origin as the AID Origin is trusted to the same degree as the AID itself. To compare, the verifier lowercases the `href` host, converts it to A-labels, and uses port 443 if none is present; the result must equal the AID Origin exactly (Appendix A.4). No other normalization is applied. Every other link **MUST** carry a `statement`, and verifiers **MUST** ignore links without a valid one (Section 7.1).

For DID-anchored agents, every link **MUST** carry a statement signed with a key usable for assertion, and verifiers **MUST** ignore links without one; otherwise whoever controls the origin could replace linked metadata without holding any key authorized by the DID.

5.5. Example

```
{
 "id": "agent://example.com/planner",
 "name": "Example Trip Planner",
 "operator": {
  "name": "Example Travel Ltd",
  "url": "https://example.com"
 },
 "keys": {
  "keys": [
   {
    "kty": "OKP",
    "crv": "Ed25519",
    "x": "11qYAYKxCrfVS_7TyWQHOg7hcvPapiMlrwIaaPcHURo",
    "kid": "kPrK_qmxVWaYVA9wwBF6Iuo3vVzz7TxHCTwXBygrS4k",
    "alg": "EdDSA",
    "use": "sig"
   }
  ]
 },
 "links": [
  {
   "rel": "agent-card",
   "href": "https://example.com/planner/agent-card.json",
   "type": "application/json"
  },
  {
   "rel": "mcp-server-card",
   "href": "https://cdn.example.net/planner/server.json",
   "statement": "eyJ0eXAiOiJhZ2VudC1zdGF0ZW1lbnQrand0Ii..."
  }
 ],
 "updated": "2026-10-20T09:00:00Z"
}
```


Figure 3: Example Agent Identity Document (origin-anchored)

5.6. Retrieval and Validation

To obtain the AID for an agent URI, a verifier:

1. Parses and canonicalizes the agent URI (Section 4.2), rejecting it on failure.
2. Constructs the AID URL (Section 4.3).
3. Applies the fetch policy of Section 11.4.
4. Sends an HTTPS GET with `Accept: application/agent-id+json`, validating the server certificate for `dns-name`.
5. Treats any status code other than 200 (OK) [RFC9110], and other than 304 (Not Modified) received in response to a conditional request revalidating a cached copy [RFC9111], as a retrieval failure. Redirects are always failures.
6. Rejects responses larger than 64 KiB, or that are not valid JSON objects. Verifiers **MUST NOT** reject a response solely because its media type is `application/json` (to support static hosting) but **MUST** reject any other media type.
7. Rejects the document unless its `id` member is identical to the canonical agent URI.
8. Performs the anchoring checks of Section 6.

Verifiers **SHOULD** cache AIDs according to HTTP caching [RFC9111]. Operators **SHOULD NOT** set a freshness lifetime longer than 24 hours, so that key removal takes effect promptly (Section 6.3).

6. Anchoring

An Agent Key is *anchored* when the verifier has established that whoever controls the agent URI published it. An AID uses exactly one of two anchoring modes.

6.1. Origin-Anchored Agents

The AID contains `keys` and no `did`. The keys are anchored by the HTTPS retrieval of Section 5.6: control of the AID Origin is control of the agent's identity.

Each JWK in `keys` **MUST** contain only public key parameters, have a `kid` equal to its Key Identifier (Section 5.2), have an `alg` member, and be usable for signatures (`"use": "sig"`, or a `key_ops` containing `"verify"`).

Implementations **MUST** support EdDSA (Ed25519) and ES256. Algorithms **MUST** be restricted as described in [RFC8725]; in particular, `none` and symmetric algorithms **MUST NOT** be accepted.

This mode needs nothing beyond static HTTPS hosting. Its weakness is that identity is exactly as durable as control of the domain (Section 11.2).

6.2. DID-Anchored Agents

The AID contains a `did` member and no `keys`. The agent's keys are the verification methods of the DID document, with the purposes defined in Section 5.3.

The binding between the agent URI and the DID **MUST** be bidirectional: the AID names the DID in `did`, and the DID document lists the canonical agent URI in its `alsoKnownAs` property [DID-CORE]. Verifiers **MUST** check both directions and **MUST** treat the agent as unverified if either is missing.

Pinning. A verifier that has previously verified an agent URI as DID-anchored and recorded its DID **MUST NOT** attribute a later AID for that agent URI to the same agent unless it names the same DID. An AID that is origin-anchored or names a different DID **MUST** be treated as a different identity (for example, by rejecting it, or by treating it as a new agent with no history), not as a key rotation.

Continuity and its limits. For a verifier that has pinned the DID, DID anchoring preserves identity across a rename or domain move: the operator publishes an AID under the new agent URI and adds that URI to `alsoKnownAs`. For a verifier with no prior state, DID anchoring gives no more assurance than origin anchoring (Section 11.1).

DID methods. A DID whose resolution depends on the same host as the agent URI (for example, `did:web` on that host) adds no independence from the origin. Operators **SHOULD NOT** use such a DID for DID anchoring, and verifiers **MUST NOT** treat it as providing assurance beyond origin anchoring. DID methods that support verifiable key-rotation history (for example, [DID-WEBVH] hosted independently of the agent URI's host) are **RECOMMENDED**. Verifiers **MUST** maintain an allowlist of DID methods they support and **MUST** reject any other method.

6.3. Lifecycle, Rotation, and Revocation

Removing a key from the AID (or DID document) revokes it: every statement it signed stops verifying once verifiers' cached copies expire. Rotation and revocation are therefore the same operation, and operators rotate as follows:

1. Publish the new key alongside the old one.
2. Re-issue all statements signed by the old key, including statements carried in AIDs copied by mirrors and registries.
3. Remove the old key once re-issued statements have propagated.

This document deliberately defines no list of retired keys: with a single way to stop trusting a key, verifiers never have to distinguish a planned retirement from a compromise. Statements about metadata that changes **SHOULD** carry an `exp` so that copies held by third parties are refreshed regularly.

If status is `"suspended"` or `"retired"`, verifiers **MUST** treat all signatures and statements from the agent as invalid. A retired agent URI **SHOULD NOT** be reassigned to a different agent.

7. Agent Statements

An Agent Statement is a JWS [RFC7515] in compact serialization, signed with an Agent Key usable for assertion, that binds the agent to a specific version of a resource such as an Agent Card. It allows protocol metadata to be served from any origin, mirrored, or included in registries without losing its link to the agent.

The JWS Protected Header **MUST** contain `typ` (`"agent-statement+jwt"`), `alg` (the algorithm of the signing key), and `kid` (the Key Identifier of the signing key). The payload is a JWT Claims Set [RFC7519] with:

`iss`: **REQUIRED**. The canonical agent URI, without a fragment.

`sub`: **REQUIRED**. The URL of the resource the statement is about.

`resource_digest`: **REQUIRED**. An object with members `alg` (`"sha-256"`), `canon` (`"jcs"` or `"none"`), and `value` (the base64url-encoded digest). With `"jcs"`, the digest is computed over the resource canonicalized per [RFC8785]; with `"none"`, over the exact bytes of the representation.

`iat`: **REQUIRED**. Issue time.

`exp`: **OPTIONAL**. Expiry time.

A statement protects only the bytes of the resource it names, not other resources that resource refers to.

7.1. Verification

To verify a statement carried in a link of an AID, a verifier:

1. Checks that `typ` is `"agent-statement+jwt"`.

2. Checks that iss is identical to the AID's id, and checks the AID's status (Section 6.3).
3. Checks that sub is identical to the link's href.
4. Selects an anchored key usable for assertion whose Key Identifier equals kid, and verifies the signature.
5. Checks iat and, if present, exp.
6. Obtains the resource at sub, either by fetching it under the policy of Section 11.4 or by using a copy, and compares its digest with resource_digest.

Statements obtained other than through an AID link (for example, from a registry) are verified in the same way, except that step 2 uses the AID retrieved for iss and step 3 compares sub with the URL the verifier intends to use.

8. Request Signing

8.1. Signing

An agent proves its identity on an HTTP request by signing it as specified in [WEB-BOT-AUTH], with the additional requirements in this section. A request signed this way is a conforming [WEB-BOT-AUTH] request.

The request **MUST** include a Signature-Agent Dictionary member, keyed by the signature label, whose value is the agent's AID URL (Section 4.3) and which carries the parameter type=agent-id:

```
Signature-Agent: sig1="https://example.com/.well-known/agent/planner"
   ;type=agent-id
```

Figure 4: Signature-Agent member (line break for display only)

The signature **MUST**:

- cover @method, @authority, @target-uri, and "signature-agent";key="<label>", and also content-digest [RFC9530] when the request has content;
- include created and expires, with expires no more than 300 seconds after created;
- include a keyid equal to the Key Identifier of the signing key; and
- include tag="web-bot-auth".

A nonce parameter **SHOULD** be included where the verifier is expected to detect replay.

```
POST /v1/bookings HTTP/1.1
Host: api.example.net
Content-Type: application/json
Content-Digest: sha-256=:...:
Signature-Agent: sig1="https://example.com/.well-known/agent/planner"
    ;type=agent-id
Signature-Input: sig1=("@method" "@authority" "@target-uri"
    "signature-agent";key="sig1" "content-digest");
    created=1792486800;expires=1792486860;
    keyid="kPrK_qmxVWaYVA9wwBF6Iuo3vVzz7TxHCTwXBygrS4k";
    tag="web-bot-auth"
Signature: sig1=:...:
```

Figure 5: A signed request (line breaks for display only)

8.2. Verification

A verifier **MUST** perform all of the following steps, and **MUST NOT** rely on a generic HTTP Message Signatures implementation to enforce them:

1. Select the signature with `tag="web-bot-auth"` whose label has a `Signature-Agent` member with `type=agent-id`. If there is none, the request does not carry an agent identity under this document.
2. Check that the member value is exactly an AID URL, without normalizing it, and derive the agent URI from it by reverse mapping (Section 4.3).
3. Check that the covered components include every component required by Section 8.1, that `expires` is no more than 300 seconds after `created`, and that the current time is between them, allowing for modest clock skew.
4. Retrieve the AID (Section 5.6) and check `status` (Section 6.3).
5. Select an anchored key usable for authentication whose Key Identifier equals `keyid`. Key lookup is keyed on the pair (agent URI, `keyid`), as required by [WEB-BOT-AUTH].
6. Verify the signature as specified in [RFC9421].
7. If the request has content, recompute its digest as specified in [RFC9530] and compare it with the covered `Content-Digest` value. A request with content and a missing, unsupported, or mismatched `Content-Digest` **MUST** be rejected, because a valid signature over `Content-Digest` does not by itself authenticate the content (Section 7.2.8 of [RFC9421]).

On success, the verifier has established that the request, including its content, was made by a holder of a key anchored to the agent URI. On failure, the verifier **MUST NOT** treat the request as coming from the named agent; it **MAY** process it as an unauthenticated request.

A [WEB-BOT-AUTH] verifier that does not support `type=agent-id` ignores the member, as that specification requires, and so treats the request as not signed by a known agent rather than failing.

9. Use with OAuth

Identity is not authorization, and this document defines no new OAuth mechanisms. It adds one rule: **only a request signature verified under Section 8 establishes agent identity.** OAuth client authentication, of any kind, authenticates an OAuth client, not an

agent. An authorization server **MUST NOT** present a client to users, or record it, as a verified agent unless the request in question carries such a signature, and the agent it attributes is always the one derived from that signature.

Consent screens are reached through front-channel redirects that an agent cannot sign. An authorization server can therefore show a verified agent identity at consent only if the agent signed a back-channel request for the same authorization request beforehand. Agents that want this **SHOULD** use Pushed Authorization Requests [RFC9126] and sign the pushed request. When a verified identity is shown, it **MUST** be the agent URI or DID, with the operator name presented as claimed by the agent (Section 11.3).

When an agent acts on behalf of a user or another agent, the delegation **SHOULD** be expressed with OAuth 2.0 Token Exchange [RFC8693], with the agent identified in the `act` claim by `"sub"` set to its canonical agent URI. As required by [RFC8693], only the current actor is relevant to access-control decisions.

Access tokens used by agents **SHOULD** be sender-constrained, for example with DPoP [RFC9449], using a key that is not an Agent Key usable for assertion. Resource servers **MAY** require a valid agent signature in addition to an access token.

10. Discovery

Dereferencing a known agent URI requires no discovery. To let others find which agents a domain operates, a domain **MAY** list them in an API catalog [RFC9727] (a linkset [RFC9264]) at `/.well-known/api-catalog`, using the agent URI as the link context and `agent-identity` links to the AIDs:

```
{
 "linkset": [
  {
   "anchor": "agent://example.com/planner",
   "agent-identity": [
    { "href": "https://example.com/.well-known/agent/planner",
     "type": "application/agent-id+json" }
   ]
  }
 ]
}
```

Figure 6: Listing agents in an API catalog

Publishing a catalog is optional (Section 12). Registries and search services that index agents **SHOULD** verify AIDs and statements themselves rather than asking clients to trust the index.

11. Security Considerations

11.1. Trust Model

For origin-anchored agents, the trust anchors are DNS and the Web PKI: anyone who can obtain a certificate for the agent URI's host and serve content there can impersonate the agent. This is the same trust model as the web itself and is appropriate for most agents.

DID anchoring reduces this dependency only for verifiers that have pinned the agent's DID (Section 6.2). On first contact, a verifier cannot tell a legitimate origin-anchored agent from an attacker who has taken over the domain and replaced a DID-anchored AID with an origin-anchored one. Even for a pinned DID, assurance holds only if verifiers rely solely on DID-authorized keys and on metadata covered by statements made with them (Section 5.4).

11.2. Handle Reassignment

If a domain lapses and is registered by someone else, the new holder can publish an AID for the same agent URIs. Verifiers that record identities for audit or reputation **SHOULD** record the Key Identifiers (and DID, when present) alongside the agent URI, and **SHOULD** treat a change of all keys for an agent as a significant event.

11.3. Operator Claims

Verification establishes possession of a key anchored to an agent URI or DID. It does not establish who operates the agent: `operator.name` and `operator.url` are supplied by the same party that publishes the AID. The owner of `attacker.example` can claim `"Example Travel Ltd"` as operator and point `operator.url` at the real company's website.

User interfaces and logs **MUST NOT** present an operator name as verified unless it has been confirmed by a separate mechanism trusted by the verifier, which this document does not define. They **SHOULD** display the authenticated agent URI host (in A-label form) or DID alongside any operator name.

11.4. Verifier-Initiated Fetches

Verification causes network requests to destinations chosen by untrusted parties: AID URLs from Signature-Agent values, statement subjects, and requests made by DID resolvers. Each of these can be used for server-side request forgery or denial of service. The following policy applies to **every** fetch made in the course of verification:

- Only HTTPS URLs are fetched, with certificate validation.
- Destinations whose host is, or resolves to, an address that the IANA IPv4 or IPv6 Special-Purpose Address Registries [RFC6890] mark as not globally reachable **MUST NOT** be fetched, nor hosts that are special-use names that do not resolve in the global DNS (for example, `localhost` or names under `local` or `internal`). Administrators **MAY** configure explicit exceptions for private deployments.
- The resolved address **MUST** be checked and then used for the connection (pinned), so that DNS changes cannot redirect the request after the check.
- HTTP redirects **MUST NOT** be followed, as in [WEB-BOT-AUTH].

- Verifiers **MUST** enforce timeouts and bound response sizes: 64 KiB for AIDs and a locally configured limit for other resources.
- Verifiers **MUST** bound the number of fetches per verification; a limit of 4 (for example, the AID, a DID resolution, and a statement subject) is **RECOMMENDED**, with a DID resolution counting as one fetch and the requests a DID resolver makes bounded separately. Verifiers **MUST NOT** recursively follow links from fetched documents.
- Verifiers **MUST** rate-limit fetches per host and cache results, including failures.

A statement whose signature is valid does not make its `sub` safe to fetch: an attacker can publish a legitimate AID under its own domain and sign a statement whose `sub` names an internal address.

11.5. Request Content and Replay

HTTP message signatures authenticate header values, not content. Verifiers **MUST** recompute `Content-Digest` over received content (Section 8.2); otherwise an attacker can change the content of a signed request while keeping the signed digest.

Signatures cover the method, target URI, and authority, which prevents replay against a different resource or service. The short `created`-`expires` window limits replay against the same resource; verifiers for non-idempotent operations **SHOULD** require and track `nonce` values.

11.6. Key Compromise

Compromise of an Agent Key lets the attacker act as the agent until the key is removed and caches expire. Operators **SHOULD** keep Agent Keys in hardware or a key management service, and **SHOULD** use separate keys for assertion and authentication (Section 5.3), so that compromise of a running agent does not let the attacker sign statements.

11.7. Cross-Origin Metadata

Requiring statements bound to the exact `href` for links outside the AID Origin (Section 5.4, Section 7.1) prevents a compromised or malicious third-party host, mirror, or registry from attaching metadata to an agent's identity. A statement only proves that the agent published the resource; it does not vouch for the truth of the resource's contents.

11.8. Confusable Names

Agent URIs can be visually similar (e.g., `agent://examp1e.example/planner` and `agent://example.example/planner`). User interfaces that display agent identities **SHOULD** display the A-label form of the host, and **SHOULD** warn when an agent URI or DID is new to the user.

12. Privacy Considerations

The considerations of [RFC6973] apply.

- **Linkability is the point, for agents.** Request signatures make an agent's traffic attributable and linkable across services. That is intended for the agent; it must not extend to the people it acts for. Agent URIs and `Signature-Agent` values **MUST NOT** encode user identifiers.

- **Per-user agents.** Deployments that create one agent per end user (e.g., agent://example.com/u/alice) make that user's activity linkable across all services the agent visits. Such deployments **SHOULD** use a shared agent identity and convey user context only through authorization tokens sent to parties entitled to it.
- **Enumeration.** API catalogs (Section 10) let anyone list a domain's agents. Domains that do not want their agents to be enumerable should not publish them in a catalog; AIDs remain retrievable by those who already know the agent URI.
- **Verifier fetches.** Fetching an AID reveals to the operator that a verifier has seen the agent. Caching reduces this signal.

13. IANA Considerations

13.1. URI Scheme Registration

IANA is requested to register the agent URI scheme in the "Uniform Resource Identifier (URI) Schemes" registry, using the template of [RFC7595]:

Scheme name: agent

Status: Permanent

Applications/protocols that use this scheme name: Identification of software agents. An agent URI is dereferenced to an Agent Identity Document as described in Section 4.3.

Contact: IESG <iesg@ietf.org>

Change controller: IETF

References: Section 4 of this document

Well-Known URI Support: The agent well-known URI (Section 13.2)

13.2. Well-Known URI Registration

IANA is requested to register the following in the "Well-Known URIs" registry [RFC8615]:

URI suffix: agent

Change controller: IETF

Specification document(s): Section 4.3 of this document

Status: permanent

Related information: Sub-paths are used: /.well-known/agent/<name-path> for named agents and /.well-known/agent/_ for a host's root agent.

13.3. Media Types

IANA is requested to register the following media types in the "Media Types" registry [RFC6838].

13.3.1. application/agent-id+json

Type name: application

Subtype name: agent-id+json

Required parameters: N/A

Optional parameters: N/A

Encoding considerations: binary; an Agent Identity Document is JSON [RFC8259] encoded in UTF-8.

Security considerations: See Section 11 of this document.

Interoperability considerations: N/A

Published specification: Section 5 of this document

Applications that use this media type: Verifiers of agent identity, such as websites, APIs, authorization servers, registries, and other agents.

Fragment identifier considerations: As specified for the `+json` structured syntax suffix in Section 3.1 of [RFC6839].

Additional information: Deprecated alias names for this type: N/A; Magic number(s): N/A; File extension(s): N/A; Macintosh file type code(s): N/A

Person & email address to contact for further information: IESG <iesg@ietf.org>

Intended usage: COMMON

Restrictions on usage: N/A

Author: See the Authors' Addresses section of this document.

Change controller: IETF

13.3.2. application/agent-statement+jwt

Type name: application

Subtype name: agent-statement+jwt

Required parameters: N/A

Optional parameters: N/A

Encoding considerations: 7bit; an Agent Statement is a JWS Compact Serialization [RFC7515], a sequence of base64url-encoded values separated by period characters. The +jwt structured syntax suffix is registered by [RFC8417].

Security considerations: See Section 11 of this document.

Interoperability considerations: N/A

Published specification: Section 7 of this document

Applications that use this media type: Agents publishing statements about their protocol metadata, and verifiers of those statements.

Fragment identifier considerations: N/A

Additional information: Deprecated alias names for this type: N/A; Magic number(s): N/A; File extension(s): N/A; Macintosh file type code(s): N/A

Person & email address to contact for further information: IESG <iesg@ietf.org>

Intended usage: COMMON

Restrictions on usage: N/A

Author: See the Authors' Addresses section of this document.

Change controller: IETF

13.4. Signature-Agent Type

Section 8 uses the Signature-Agent member parameter value type=agent-id. [WEB-BOT-AUTH] does not currently establish a registry of type values. This document asks that such a registry be created and that agent-id be registered in it, referencing Section 8 of this document (see Appendix B).

13.5. Link Relation

IANA is requested to register the following in the "Link Relation Types" registry [RFC8288]:

Relation Name: agent-identity

Description: Refers to the Agent Identity Document of the link context.

Reference: Section 10 of this document

Registration of agent-card and mcp-server-card is an open issue (Appendix B).

13.6. JSON Web Key Parameter

IANA is requested to register the following in the "JSON Web Key Parameters" registry [RFC7517]:

Parameter Name: agent_use

Parameter Description: Purposes for which an Agent Key may be used

Used with "kty" Value(s): *

Parameter Information Class: Public

Change Controller: IETF

Specification Document(s): Section 5.3 of this document

13.7. JSON Web Token Claim

IANA is requested to register the following in the "JSON Web Token Claims" registry [RFC7519]:

Claim Name: resource_digest

Claim Description: Digest of the resource identified by the sub claim

Change Controller: IETF

Specification Document(s): Section 7 of this document

14. References

14.1. Normative References

[CID] Sporny, M. and M. Jones, "Controlled Identifiers v1.0", W3C Recommendation, May 2025, <https://www.w3.org/TR/cid-1.0/>.

[DID-CORE] Sporny, M., Longley, D., Sabadello, M., Reed, D., Steele, O., and C. Allen, "Decentralized Identifiers (DIDs) v1.0", W3C Recommendation, July 2022, <https://www.w3.org/TR/did-core/>.

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/rfc/rfc2119>.

[RFC3339] Klyne, G. and C. Newman, "Date and Time on the Internet: Timestamps", RFC 3339, DOI 10.17487/RFC3339, July 2002, <https://www.rfc-editor.org/rfc/rfc3339>.

[RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, DOI 10.17487/RFC3986, January 2005, <https://www.rfc-editor.org/rfc/rfc3986>.

[RFC5234] Crocker, D., Ed. and P. Overell, "Augmented BNF for Syntax Specifications: ABNF", STD 68, RFC 5234, DOI 10.17487/RFC5234, January 2008, <https://www.rfc-editor.org/rfc/rfc5234>.

[RFC5890] Klensin, J., "Internationalized Domain Names for Applications (IDNA): Definitions and Document Framework", RFC 5890, DOI 10.17487/RFC5890, August 2010, <https://www.rfc-editor.org/rfc/rfc5890>.

[RFC5891] Klensin, J., "Internationalized Domain Names in Applications (IDNA): Protocol", RFC 5891, DOI 10.17487/RFC5891, August 2010, <https://www.rfc-editor.org/rfc/rfc5891>.

[RFC6838] Freed, N., Klensin, J., and T. Hansen, "Media Type Specifications and Registration Procedures", BCP 13, RFC 6838, DOI 10.17487/RFC6838, January 2013, <https://www.rfc-editor.org/rfc/rfc6838>.

[RFC6890] Cotton, M., Vegoda, L., Bonica, R., Ed., and B. Haberman, "Special-Purpose IP Address Registries", BCP 153, RFC 6890, DOI 10.17487/RFC6890, April 2013, <https://www.rfc-editor.org/rfc/rfc6890>.

[RFC7515] Jones, M., Bradley, J., and N. Sakimura, "JSON Web Signature (JWS)", RFC 7515, DOI 10.17487/RFC7515, May 2015, <https://www.rfc-editor.org/rfc/rfc7515>.

[RFC7517] Jones, M., "JSON Web Key (JWK)", RFC 7517, DOI 10.17487/RFC7517, May 2015, <https://www.rfc-editor.org/rfc/rfc7517>.

[RFC7519] Jones, M., Bradley, J., and N. Sakimura, "JSON Web Token (JWT)", RFC 7519, DOI 10.17487/RFC7519, May 2015, <https://www.rfc-editor.org/rfc/rfc7519>.

[RFC7595] Thaler, D., Ed., Hansen, T., and T. Hardie, "Guidelines and Registration Procedures for URI Schemes", BCP 35, RFC 7595, DOI 10.17487/RFC7595, June 2015, <https://www.rfc-editor.org/rfc/rfc7595>.

[RFC7638] Jones, M. and N. Sakimura, "JSON Web Key (JWK) Thumbprint", RFC 7638, DOI 10.17487/RFC7638, September 2015, <https://www.rfc-editor.org/rfc/rfc7638>.

[RFC8037] Liusvaara, I., "CFRG Elliptic Curve Diffie-Hellman (ECDH) and Signatures in JSON Object Signing and Encryption (JOSE)", RFC 8037, DOI 10.17487/RFC8037, January 2017, <https://www.rfc-editor.org/rfc/rfc8037>.

[RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <https://www.rfc-editor.org/rfc/rfc8174>.

[RFC8259] Bray, T., Ed., "The JavaScript Object Notation (JSON) Data Interchange Format", STD 90, RFC 8259, DOI 10.17487/RFC8259, December 2017, <https://www.rfc-editor.org/rfc/rfc8259>.

[RFC8288] Nottingham, M., "Web Linking", RFC 8288, DOI 10.17487/RFC8288, October 2017, <https://www.rfc-editor.org/rfc/rfc8288>.

[RFC8615] Nottingham, M., "Well-Known Uniform Resource Identifiers (URIs)", RFC 8615, DOI 10.17487/RFC8615, May 2019, <https://www.rfc-editor.org/rfc/rfc8615>.

[RFC8693] Jones, M., Nadalin, A., Campbell, B., Ed., Bradley, J., and C. Mortimore, "OAuth 2.0 Token Exchange", RFC 8693, DOI 10.17487/RFC8693, January 2020, <https://www.rfc-editor.org/rfc/rfc8693>.

[RFC8725] Sheffer, Y., Hardt, D., and M. Jones, "JSON Web Token Best Current Practices", BCP 225, RFC 8725, DOI 10.17487/RFC8725, February 2020, <https://www.rfc-editor.org/rfc/rfc8725>.

[RFC8785] Rundgren, A., Jordan, B., and S. Erdtman, "JSON Canonicalization Scheme (JCS)", RFC 8785, DOI 10.17487/RFC8785, June 2020, <https://www.rfc-editor.org/rfc/rfc8785>.

[RFC9110] Fielding, R., Ed., Nottingham, M., Ed., and J. Reschke, Ed., "HTTP Semantics", STD 97, RFC 9110, DOI 10.17487/RFC9110, June 2022, <https://www.rfc-editor.org/rfc/rfc9110>.

[RFC9111] Fielding, R., Ed., Nottingham, M., Ed., and J. Reschke, Ed., "HTTP Caching", STD 98, RFC 9111, DOI 10.17487/RFC9111, June 2022, <https://www.rfc-editor.org/rfc/rfc9111>.

[RFC9421] Backman, A., Ed., Richer, J., Ed., and M. Sporny, "HTTP Message Signatures", RFC 9421, DOI 10.17487/RFC9421, February 2024, <https://www.rfc-editor.org/rfc/rfc9421>.

[RFC9530] Polli, R. and L. Pardue, "Digest Fields", RFC 9530, DOI 10.17487/RFC9530, February 2024, <https://www.rfc-editor.org/rfc/rfc9530>.

[RFC9651] Nottingham, M. and P. Kamp, "Structured Field Values for HTTP", RFC 9651, DOI 10.17487/RFC9651, September 2024, <https://www.rfc-editor.org/rfc/rfc9651>.

[WEB-BOT-AUTH] Meunier, T. and S. Major, "HTTP Message Signatures for automated traffic", Work in Progress, Internet-Draft, draft-ietf-webbotauth-httpsig-protocol-00, 1 September 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-webbotauth-httpsig-protocol-00>.

14.2. Informative References

[A2A] A2A Project, Linux Foundation, "Agent2Agent (A2A) Protocol Specification", 2025, <https://a2a-protocol.org/latest/specification/>.

[DID-WEBVH] Decentralized Identity Foundation, "The did:webvh DID Method", 2025, <https://identity.foundation/didwebvh/>.

[MCP] Model Context Protocol Project, "Model Context Protocol Specification", 2025, <https://modelcontextprotocol.io/specification/>.

[RFC6839] Hansen, T. and A. Melnikov, "Additional Media Type Structured Syntax Suffixes", RFC 6839, DOI 10.17487/RFC6839, January 2013, <https://www.rfc-editor.org/rfc/rfc6839>.

[RFC6973] Cooper, A., Tschofenig, H., Aboba, B., Peterson, J., Morris, J., Hansen, M., and R. Smith, "Privacy Considerations for Internet Protocols", RFC 6973, DOI 10.17487/RFC6973, July 2013, <https://www.rfc-editor.org/rfc/rfc6973>.

[RFC8417] Hunt, P., Ed., Jones, M., Denniss, W., and M. Ansari, "Security Event Token (SET)", RFC 8417, DOI 10.17487/RFC8417, July 2018, <https://www.rfc-editor.org/rfc/rfc8417>.

[RFC9126] Lodderstedt, T., Campbell, B., Sakimura, N., Tonge, D., and F. Skokan, "OAuth 2.0 Pushed Authorization Requests", RFC 9126, DOI 10.17487/RFC9126, September 2021, <https://www.rfc-editor.org/rfc/rfc9126>.

[RFC9264] Wilde, E. and H. Van de Sompel, "Linkset: Media Types and a Link Relation Type for Link Sets", RFC 9264, DOI 10.17487/RFC9264, July 2022, <https://www.rfc-editor.org/rfc/rfc9264>.

[RFC9449] Fett, D., Campbell, B., Bradley, J., Lodderstedt, T., Jones, M., and D. Waite, "OAuth 2.0 Demonstrating Proof of Possession (DPoP)", RFC 9449, DOI 10.17487/RFC9449, September 2023, <https://www.rfc-editor.org/rfc/rfc9449>.

[RFC9727] Smith, K., "api-catalog: A Well-Known URI and Link Relation to Help Discovery of APIs", RFC 9727, DOI 10.17487/RFC9727, June 2025, <https://www.rfc-editor.org/rfc/rfc9727>.

Appendix A. Test Vectors: Agent URI Processing

This appendix is informative. It gives expected results for the syntactic processing defined in Section 4 and Section 5.4, so that independent implementations can check that they agree.

A.1. Parsing and Canonicalization

Each vector shows an input, then its result. "valid" inputs are canonical. "normalized" inputs differ from the ABNF only by deviations listed in Section 4.2; parsers **MAY** accept them and, if they do, **MUST** produce the canonical form shown. "reject" inputs **MUST** be rejected. `<U+00FC>` denotes the single character LATIN SMALL LETTER U WITH DIAERESIS.

```
agent://example.com/planner
   valid       agent://example.com/planner
agent://planner.example.com
   valid       agent://planner.example.com    (root agent)
agent://example.com/acme/billing-assistant
   valid       agent://example.com/acme/billing-assistant
agent://123.example/planner
   valid       agent://123.example/planner
agent://example.com/p#kPrK_qmxVWaYVA9wwBF6Iuo3vVzz7TxHCTwXBygrS4k
   valid       agent://example.com/p          (plus key fragment)

AGENT://Example.COM/planner
   normalized  agent://example.com/planner
agent://example.com:443/planner
   normalized  agent://example.com/planner
agent://example.com:/planner
   normalized  agent://example.com/planner
agent://example.com/planner/
   normalized  agent://example.com/planner
agent://planner.example.com/
   normalized  agent://planner.example.com
agent://b<U+00FC>cher.example/planner
   normalized  agent://xn--bcher-kva.example/planner

agent://192.168.0.1/planner            reject  all-digit final label
agent://localhost/planner              reject  single-label host
agent://[::1]/planner                  reject  IP literal
agent://user@example.com/planner       reject  userinfo
agent://example.com./planner           reject  trailing "." on host
agent://example.com:8443/planner       reject  port
agent://example.com:0443/planner       reject  port
agent://example.com/planner?task=1     reject  query
agent://example.com/Planner            reject  uppercase name-path
agent://example.com/plan%6Eer          reject  percent-encoding
agent://example.com/a/../planner       reject  dot-segment
agent://example.com//planner           reject  empty segment
agent://example.com/-planner           reject  leading "-"
agent://example.com/_                  reject  reserved segment
agent://example.com/a/b/c/d/e/f/g/h/i  reject  more than 8 segments
agent+https://example.com/planner      reject  not the agent scheme
```

Figure 7: Parsing and canonicalization vectors

`agent://example.com/p` with and without the key fragment identify the same agent (Section 4.2).

A.2. Mapping to AID URLs

```
agent://example.com/planner
   https://example.com/.well-known/agent/planner
agent://planner.example.com
   https://planner.example.com/.well-known/agent/_
agent://example.com/acme/billing-assistant
   https://example.com/.well-known/agent/acme/billing-assistant
agent://xn--bcher-kva.example/planner
   https://xn--bcher-kva.example/.well-known/agent/planner
```

Figure 8: Agent URI to AID URL vectors

A.3. Reverse Mapping of Signature-Agent Values

Values are compared exactly, without normalization (Section 8.2).

```
https://example.com/.well-known/agent/planner
   agent://example.com/planner
https://planner.example.com/.well-known/agent/_
   agent://planner.example.com
https://example.com:8443/.well-known/agent/planner
   reject  port
https://Example.com/.well-known/agent/planner
   reject  not canonical (host case)
https://example.com:443/.well-known/agent/planner
   reject  not canonical (default port)
https://example.com/.well-known/agent/planner/
   reject  not canonical (trailing "/")
https://example.com/.well-known/agent/planner?v=1
   reject  query
http://example.com/.well-known/agent/planner
   reject  not HTTPS
https://example.com/.well-known/agent
   reject  neither a name-path nor the reserved segment
https://example.com/.well-known/agent/_/planner
   reject  reserved segment inside a name-path
https://example.com/.well-known/agent/Planner
   reject  invalid name-path
https://example.com/.well-known/agent/planner#k
   reject  fragment
https://b<U+00FC>cher.example/.well-known/agent/planner
   reject  not canonical (U-label host)
```

Figure 9: Reverse mapping vectors

A.4. AID Origin Comparison

For agent://example.com/planner, the AID Origin is ("https", "example.com", 443). Link targets compare as follows (Section 5.4):

```
https://example.com/planner/agent-card.json          same origin
https://example.com:443/planner/agent-card.json      same origin
https://EXAMPLE.com/planner/agent-card.json          same origin
https://planner.example.com/agent-card.json          different host
https://example.com:8443/planner/agent-card.json     different port
https://example.com./planner/agent-card.json         different host
http://example.com/planner/agent-card.json           invalid href
```

Figure 10: AID Origin comparison vectors

A link whose `href` is not an HTTPS URL is invalid regardless of origin, and a statement cannot make it valid (Section 5.4).

Appendix B. Open Issues

This appendix will be removed before publication.

1. **Signature-Agent type registry.** [WEB-BOT-AUTH] has no registry for `type` values. Work with the webbotauth WG to create one, or to define `agent-id` there. Until then, verifiers that only implement [WEB-BOT-AUTH] ignore agent identities.
2. **Back-references from protocol metadata.** Agree with the A2A and MCP projects on how an Agent Card or MCP server card names its agent URI, and who registers the `agent-card` and `mcp-server-card` link relations.
3. **Companion documents.** Credentials for short-lived agent instances (in -04, a profile of the WIMSE Workload Identity Token) and bindings of agent keys to OAuth client authentication (in -04, `private_key_jwt` attribution) are to be specified separately, in coordination with the WIMSE and OAuth working groups.
4. **Cryptographic test vectors.** Appendix A covers URI processing. Once an implementation can produce and cross-check them, add vectors for statement `iss`/`sub` binding, DID key conversion and per-purpose selection, `agent_use` enforcement, content tampering with a valid signature, fetches to forbidden destinations, and expired statements.

Appendix C. Changes from draft-narvaneni-agent-uri-04

This revision simplifies the document so that each task has one mechanism.

Removed or moved out:

- **Ports.** Agent URIs no longer contain ports, so an agent's identity depends only on its host and name.
- **Instance credentials.** The profile of the WIMSE Workload Identity Token moves to a companion document (Appendix B).
- **OAuth client-authentication attribution.** The association check, qualifying-key rules, and the `agent_uri` client metadata parameter are removed. Only a verified request signature establishes agent identity (Section 9).
- The appendix listing changes from draft-narvaneni-agent-uri-03.

Added or completed:

- Full IANA registration templates.
- "One way to do each thing" as a design principle.

Closed open issues:

- **Retired keys:** the re-issue model is kept, giving a single way to stop trusting a key (Section 6.3).
- **DNS-based verification:** not adopted. Any operator that can obtain a TLS certificate for a domain can serve AIDs from static hosting, and a DNS alternative would need DNSSEC and a second verification path.
- **Reserved root segment and name-path rules:** kept as specified.
- **Ports:** removed.

Acknowledgments

The authors thank the reviewers of earlier revisions, whose feedback shaped this document.

Authors' Addresses

Yaswanth Narvaneni
Independent Researcher
London
United Kingdom
Email: yaswanth+ietf@gmail.com

Sai Purnima Ravi
Independent Researcher
Email: r.sai.purnima+ietf@gmail.com